Secure Notes & Export System

Assignment I: Secure Application Design, Implementation & CI/CD Verification

Department of Computer Science, Aarhus University

Field	Details
Author	Utku Balikci
Course	Software Security (Assignment I)
Date	October 2026
System	Python 3.13 / FastAPI: three services and one external package
Repository	https://github.com/utku-balikci/Software_sec_assigment1
Verification	Bandit, Pytest, Jenkins LTS pipeline, local pipeline runner

Contents

Summary 4
1. System Design 5
1.1 Domain and Rationale 5
1.2 Assets 5
1.3 Components and Trust Boundaries 5
1.4 Functionality 6
1.5 Access Control Matrix 6
2. Authority Flow 6
2.1 Where Authority Is Established 6
2.2 Token Binding 6
2.3 Export Sequence 7
2.4 Complete Mediation 7
2.5 Fail-Safe Defaults 7
3. STRIDE Threat Model 8
4. Planted Vulnerabilities 9
4.1 CWE-798: Hard-coded Credentials 10
4.2 CWE-862: Missing Authorization (IDOR) 10
4.3 CWE-209: Sensitive Information Exposure 10
4.4 Compositional Flaw: Confused Deputy 10
5. Secure Design Principles 10
6. CI/CD Pipeline and Gates 11
6.1 Stages and Gate Decisions 11
6.2 Build Evidence 11
6.3 Functional and Authorization Tests 13
7. Experiment A: Supply Chain Integrity Gate 13
7.1 Problem 13
7.2 Consumer-Controlled Manifest 13
7.3 Verifier and Gate 14
7.4 Tamper Demonstration 14
7.5 Scope 15
8. What Would Change for Production 15
9. Known Limitations 15
10. Conclusion 16
11. AI-Use Declaration 16
References 16
Appendix A: Git History 18
Appendix B: Test Suite 18
Appendix C: Jenkinsfile 18
Appendix D: Repository Layout 19
Appendix E: Reproduction 19

Summary

This report documents the design, implementation and verification of a small multi-tier application for private note management and HTML export. The system runs as three independent FastAPI services and uses an external formatting package installed as a wheel. It deliberately contains four vulnerabilities: three from the course catalogue (CWE-798, CWE-862, CWE-209) and one compositional Confused Deputy flaw. Each can be exploited in vulnerable mode and blocked in secure mode, so the effect of every control is visible.

A Jenkins pipeline runs static analysis and functional tests on each build. Experiment A adds a consumer-controlled integrity check that rejects a substituted dependency before it is installed.

1. System Design

1.1 Domain and Rationale

A notes system with tenants, an administrator role and an export feature exercises the properties this course cares about. Data must stay partitioned between users, destructive actions must be restricted, and one workflow must cross service boundaries without any service receiving more authority than its job requires. The export feature also brings in a third-party formatter, which lets us test whether a dependency can be contained once it is running.

1.2 Assets

Asset	Owner	Objective	Impact if compromised
Private notes	Each tenant	Confidentiality, integrity	Disclosure of another tenant's records (CWE-862)
JWT signing key	API gateway	Authenticity	Authentication bypass for every account (CWE-798)
Environment and in-memory data	System	Confidentiality	Leakage of configuration and all notes (CWE-209)
Exported HTML	Requesting tenant	Confidentiality	Disclosure through an unauthenticated worker call

1.3 Components and Trust Boundaries

Three services run as separate processes on different ports. Each has one responsibility and one failure domain.

Component	Port	Responsibility	Security role
Application API (C1)	8000	Login, token issue and verification, routing	Establishes caller identity
Resource Service (C2)	8001	Note storage and record lookup	Enforces ownership on every record
Processing Service (C3)	8002	Export job and HTML rendering	Reads one note per job; no write or delete rights
html_note_formatter	Wheel	Wraps note text in HTML	External dependency, run inside C3

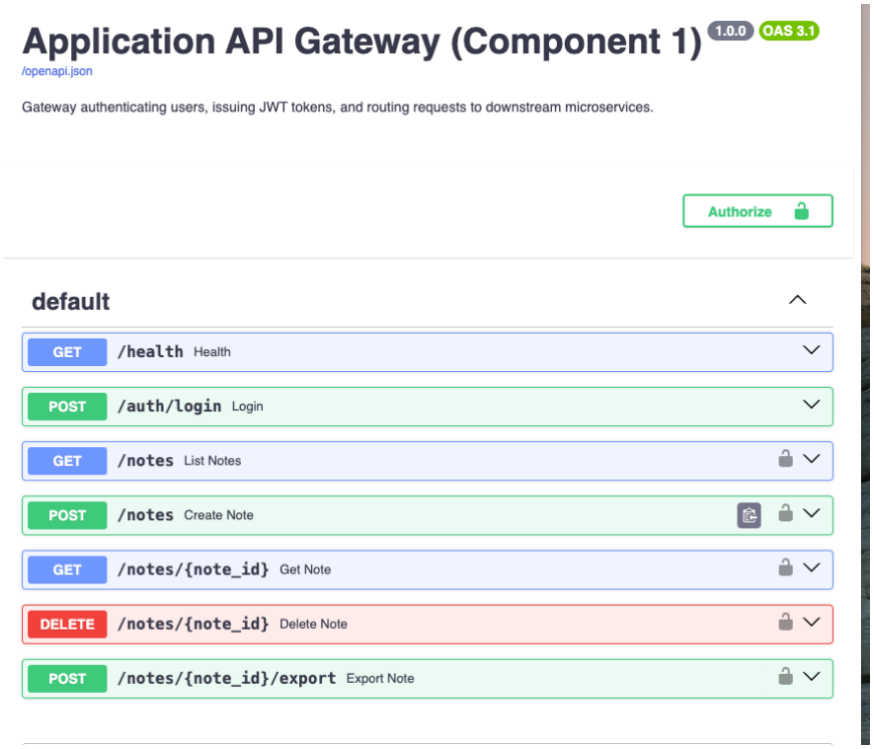


Figure 1. *Swagger UI of the Application API gateway (http://127.0.0.1:8000/docs), listing the public and authenticated endpoints.*

1.4 Functionality

Three functions are implemented. Creating a note travels from the client through C1 to C2, which stores it under the caller's identity. Viewing a note follows the same path, but C2 returns it only to its owner or to an administrator. Exporting a note touches all three components: C1 authenticates the request and hands it to C3, which fetches the note from C2 using the caller's identity, passes it to the formatter, and returns the HTML.

1.5 Access Control Matrix

Account	Alice's notes	Bob's notes	Export	Delete
Alice (user): alice / password123	Read, write	Denied	Own notes	Denied (403)
Bob (user): bob / password123	Denied	Read, write	Own notes	Denied (403)
Charlie (admin): charlie / admin123	Read only	Read only	All notes	Allowed (200)

2. Authority Flow

2.1 Where Authority Is Established

Authority is established in three places. The Application API establishes identity: it is the only component that receives passwords, and on a successful login it issues an HMAC-SHA256 signed token. It then forwards the verified identity to the internal services as the X-User and X-Role headers. The Resource Service establishes ownership: it records the caller as owner when a note is created and checks that owner on every later access. Dependency trust is established at build time, when the consumer's manifest records the approved digest of the formatter wheel and the pipeline checks it before installation (Experiment A).

2.2 Token Binding

The token carries three claims: the subject, the role and an expiry one hour after issue. The payload is built in `services/app_api/main.py` as follows.

```
def create_access_token(data: dict) -> str:
    payload = data.copy()
    payload.update({'exp': time.time() + 3600})
    return jwt.encode(payload, JWT_SECRET_KEY, algorithm=JWT_ALGORITHM)
```

The token has no audience claim and no per-resource scopes. The Resource Service therefore makes resource and action decisions from the forwarded headers, not from the token. This is a simplification, and Section 9 returns to it.

2.3 Export Sequence

Figure 2 traces one export request. Every arrow that crosses a trust boundary carries a check: C1 verifies the token, C3 requires the forwarded identity, and C2 enforces ownership before it returns any note data.

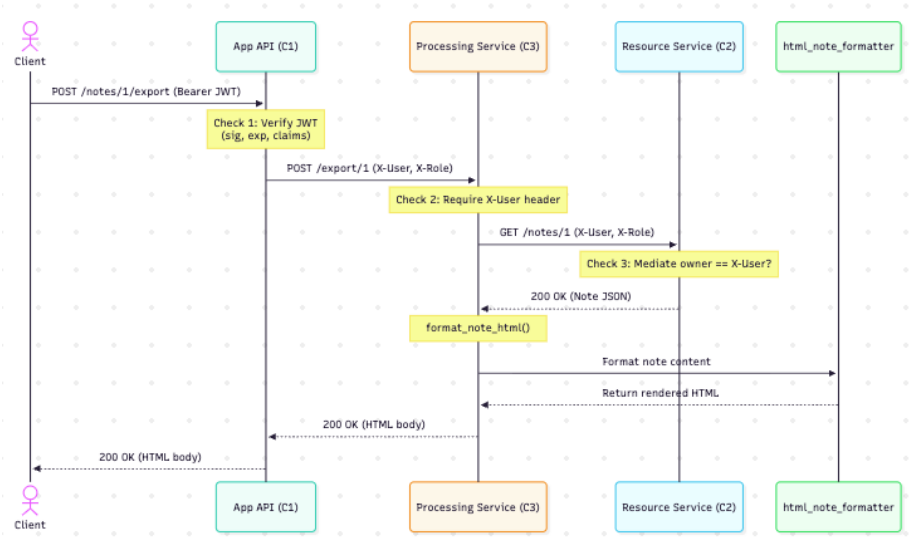


Figure 2. *Sequence of an export request across C1, C3, C2 and the formatter, with the authorization check at each boundary.*

2.4 Complete Mediation

Complete mediation means every access is checked where the data lives, not only at the perimeter. The Resource Service therefore checks ownership on each lookup:

```
# services/resource_service/main.py
if x_role != 'admin' and note['owner'] != x_user:
    raise HTTPException(status_code=403,
                        detail='Forbidden: You do not own this note')
```

A valid user who guesses another tenant's note ID is refused for the same reason.

2.5 Fail-Safe Defaults

Every layer denies when information is missing. A request with no Authorization header is refused with 401 before any downstream service is contacted (Figure 3). Internal calls without X-User are also refused with 401. Deleting a note requires the admin role, so an ordinary user receives 403 even for a note they own. Unknown note IDs return 404 without revealing the stored schema.

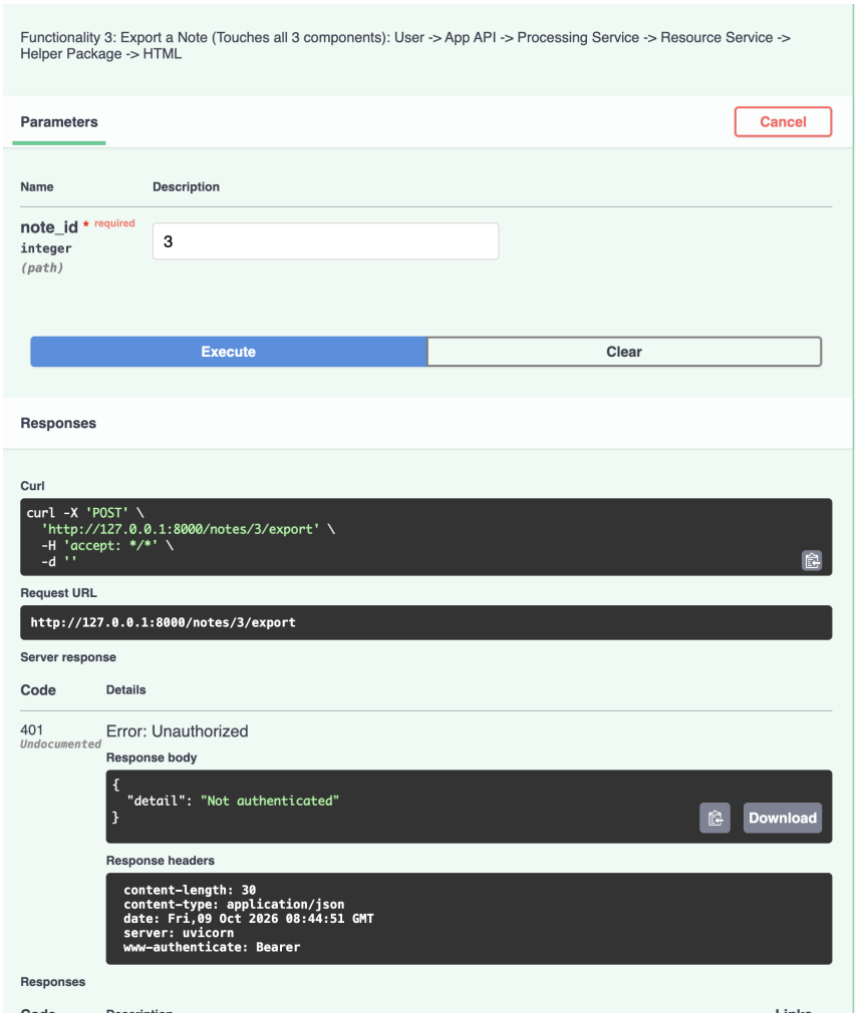


Figure 3. *Unauthenticated export request (POST /notes/3/export) rejected with 401 Unauthorized before contacting downstream services.*

3. STRIDE Threat Model

The table below applies STRIDE to the system. The Status column separates controls that are tested, controls that are implemented but not tested, and controls that exist only as planned production measures.

Category	Threat	Component	Mitigation	Status
Spoofing	Forged token or spoofed X-User header	Gateway; internal services	Signed JWT verified at the gateway; identity headers trusted only on the internal network	Implemented and tested (test_demonstrate_cwe_798_hardcoded_credentials)
Tampering	Note or export parameters altered	C1 to C2 and C3	Pydantic schema validation; TLS planned	Implemented, untested

Category	Threat	Component	Mitigation	Status
	between services			
Repudiation	User denies deleting or exporting a note	C2, C1	Audit log of caller, time and note ID	Planned (only terminal output exists)
Information disclosure	Tenant reads another tenant's note (IDOR)	C2, GET /notes/{id}	Owner check on every access	Implemented and tested (test_functionality_2_view_note_and_authorization; test_demonstrate_cwe_862_idor)
Denial of service	Oversized or repeated export requests	C3	Payload limits, timeouts, job queue	Planned
Elevation of privilege	Ordinary user calls admin delete	C1, C2	Role check (role == admin)	Implemented and tested (test_admin_and_delete_permissions)

4. Planted Vulnerabilities

All four flaws sit behind the VULNERABLE_MODE toggle. Figure 4 shows the three services starting in vulnerable mode. Figure 5 shows all four exploits succeeding against that configuration, and Figure 6 shows the same four attacks blocked in secure mode.

```
Starting Secure Notes & Export Microservices Architecture...
  >>> MODE: [VULNERABLE MODE ACTIVE] (Planted Flaws Enabled) <<<
================================================================
[+] Launching Resource Service (Port 8001)...
INFO:     Started server process [73173]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
[+] Launching Processing Service (Port 8002)...
INFO:     Started server process [73174]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8002 (Press CTRL+C to quit)
[+] Launching App API Gateway (Port 8000)...
INFO:     Started server process [73175]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

All 3 services are active!
API Gateway Swagger Docs: http://127.0.0.1:8000/docs
```

Figure 4. *Services starting in vulnerable mode (python run_services.py --vulnerable).*

```
============================= test session starts ==============================
platform darwin -- Python 3.13.9, pytest-8.4.2, pluggy-1.5.0 -- /opt/anaconda3/bin/python
cachedir: .pytest_cache
rootdir: /Users/utkubalikci/Desktop/Software_sec_assigment1
plugins: anyio-4.10.0
collected 4 items

tests/test_vulnerabilities.py::test_demonstrate_cwe_798_hardcoded_credentials
[+] [EXPLOIT SUCCEEDED] CWE-798: Forged admin JWT accepted by Gateway!
PASSED
tests/test_vulnerabilities.py::test_demonstrate_cwe_862_idor
[+] [EXPLOIT SUCCEEDED] CWE-862: Bob successfully accessed Alice's note via IDOR!
PASSED
tests/test_vulnerabilities.py::test_demonstrate_cwe_209_sensitive_info_exposure
[+] [EXPLOIT SUCCEEDED] CWE-209: Debug endpoint leaked server environment and notes database!
PASSED
tests/test_vulnerabilities.py::test_demonstrate_compositional_vulnerability
[+] [EXPLOIT SUCCEEDED] Compositional Flaw: Unauthenticated caller forced export via Confused Dep
uty!
PASSED

============================== 4 passed in 0.15s ===============================
```

Figure 5. *Vulnerable-mode test run: all four exploits succeed.*

4.1 CWE-798: Hard-coded Credentials

The gateway signs tokens with a constant string committed to the repository (`services/app_api/main.py`, lines 20 to 35). Anyone who reads the code can mint a token for `charlie` with role `admin`, and the gateway accepts it. In secure mode the key is read from `JWT_SECRET_KEY`, and the forged token is rejected with 401. The fallback value in the secure branch is a residual weakness; Section 9 covers it.

4.2 CWE-862: Missing Authorization (IDOR)

`GET /notes/{note_id}` fetches a record by its integer key and never compares the owner with the caller. Bob logs in legitimately, requests note 1 and receives Alice's note. Secure mode performs the owner check and returns 403.

4.3 CWE-209: Sensitive Information Exposure

`GET /debug/environment` in `services/resource_service/main.py` returns the process environment and the full in-memory note store to any caller. Secure mode suppresses the route and returns 403.

4.4 Compositional Flaw: Confused Deputy

Each service looks reasonable on its own, but their combination is not. In vulnerable mode, C3 accepts calls on port 8002 without caller identity and fetches notes from C2 with an admin role. An attacker who bypasses the gateway therefore receives Alice's export. Secure mode requires X-User, forwards the caller's own identity, and refuses the request with 401 when it is missing.

```
============================= test session starts ==============================
platform darwin -- Python 3.13.9, pytest-9.1.1, pluggy-1.6.0 -- /Users/utkubalikci/Desktop/Software_sec_assigment1/venv/bin/python3
cachedir: .pytest_cache
rootdir: /Users/utkubalikci/Desktop/Software_sec_assigment1
plugins: anyio-4.15.1
collected 4 items

tests/test_vulnerabilities.py::test_demonstrate_cwe_798_hardcoded_credentials
[+] [DEFENSE ACTIVE] CWE-798: Forged JWT rejected (401 Unauthorized)
PASSED
tests/test_vulnerabilities.py::test_demonstrate_cwe_862_idor
[+] [DEFENSE ACTIVE] CWE-862: IDOR blocked by Resource Service (403 Forbidden)
PASSED
tests/test_vulnerabilities.py::test_demonstrate_cwe_209_sensitive_info_exposure
[+] [DEFENSE ACTIVE] CWE-209: Debug endpoint protected (403 Forbidden)
PASSED
tests/test_vulnerabilities.py::test_demonstrate_compositional_vulnerability
[+] [DEFENSE ACTIVE] Compositional Flaw: Processing Service rejected unauthenticated request (401)
PASSED

============================== 4 passed in 0.11s ===============================
```

Figure 6. *Secure-mode test run: all four defences active (pytest -s -v tests/test_vulnerabilities.py).*

5. Secure Design Principles

Three principles from the course are implemented and tested. The table below links each to its design decision, its code and its test.

Principle	Design decision	Implementation	Test or evidence
Least privilege	Tenants see only their own notes; deletion is admin-only; C3 holds no write or delete rights	Owner filter on list; role check in delete_note	test_admin_and_delete_permissions: Alice gets 403, Charlie gets 200
Complete mediation	Every request is checked at the service that owns the data	get_note asserts owner == X-User; gateway verifies the token on every call	test_functionality_2_view_note_and_authorization: Bob gets 403 for Alice's note
Fail-safe defaults	Missing credentials produce a refusal, never access	401 on missing or expired token and on missing X-User; 404 for unknown notes	Figure 3: export without a token returns 401

6. CI/CD Pipeline and Gates

6.1 Stages and Gate Decisions

The Jenkinsfile defines five stages, summarised in the table below. The integrity gate and the test stage are release-blocking: a digest mismatch or a failing test stops the build. The Bandit stage is informational in the current Jenkinsfile, because it ends with `|| true`. Bandit still reports every finding, but the build does not fail on them. We chose this because the current scan contains no Medium or High findings. Enforcing a High-severity threshold would be a one-line change.

Stage	Activity	Decision
1. Checkout and setup	Clone, create virtual environment, install dependencies	Build succeeds
2. Package integrity gate	Lock manifest (chmod 444), build wheel, verify SHA-256	Blocks on mismatch
3. Static analysis	Bandit over services/	Informational (`|| true`)
4. Integration tests	Start services, run test_flow.py, write JUnit XML	Blocks on failure
5. Post actions	Publish JUnit report, archive results and docs	Records evidence

6.2 Build Evidence

Jenkins Build #2 (commit 05087a6) completed all stages in 14 seconds (Figure 7). Its test dashboard shows five passing tests in 59 ms (Figure 8). That build predates the integrity gate, so its stage view does not include it. The local runner `run_pipeline_local.sh`, which mirrors the Jenkinsfile, scanned 292 lines and reported no Medium or High findings and seven Low-severity findings (Figure 9).

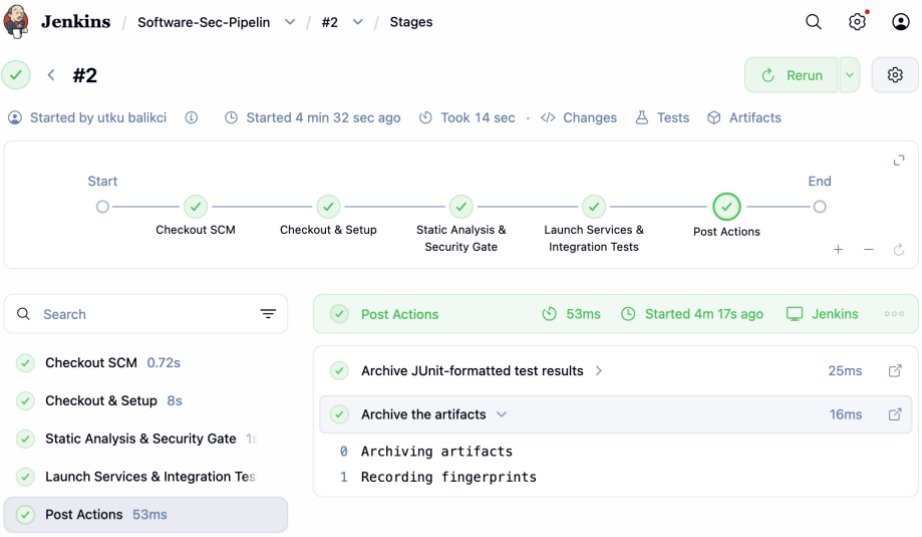


Figure 7. *Jenkins Build #2 stage view: all stages green, completed in 14 seconds.*

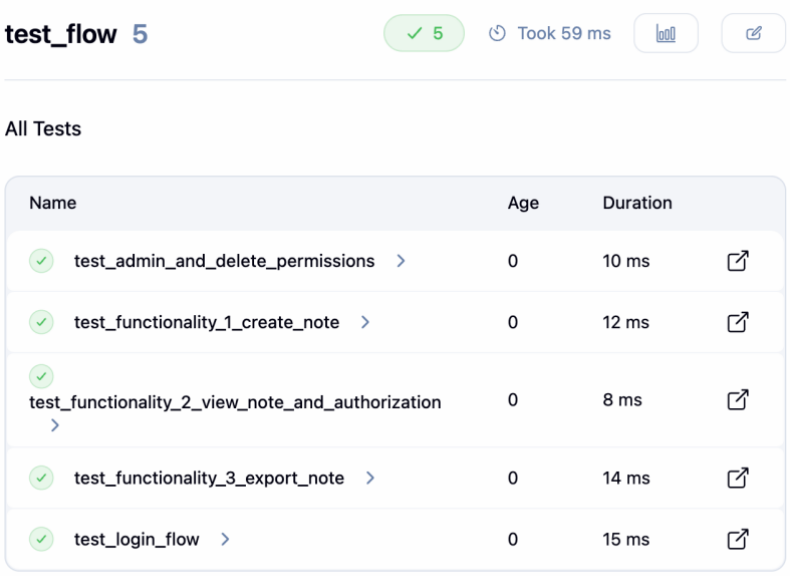


Figure 8. *Jenkins test dashboard for test_flow: 5 of 5 tests passed in 59 ms.*

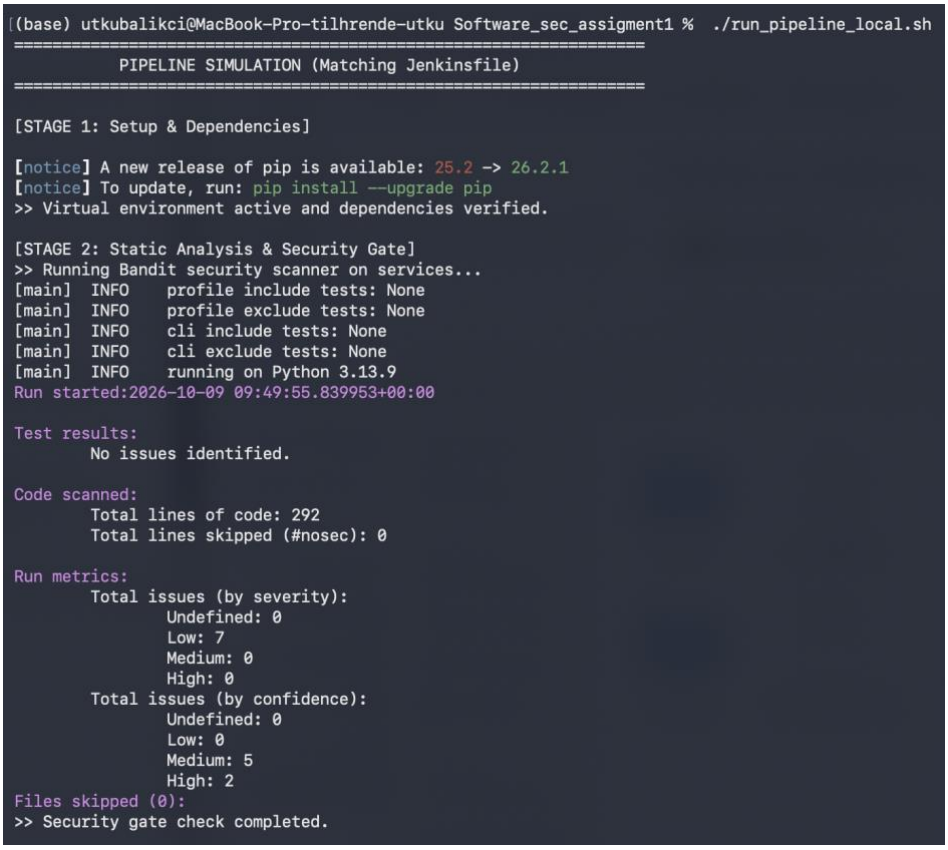


Figure 9. *Local pipeline run (run_pipeline_local.sh): Bandit reports no Medium or High findings in services/.*

6.3 Functional and Authorization Tests

`tests/test_flow.py` contains five tests: login for all three accounts; note creation with ownership; viewing with cross-user denial; the three-service export; and admin-only deletion. The local run passes all five in 0.22 seconds (Figure 10).



Figure 10. *Local pytest run of tests/test_flow.py: 5 of 5 passed in 0.22s.*

7. Experiment A: Supply Chain Integrity Gate

7.1 Problem

A supplier can replace the bytes of a wheel while keeping the same name and version. `pip` installs the changed file without complaint, so the version string alone proves nothing about the content.

7.2 Consumer-Controlled Manifest

The consumer keeps an approval manifest in `manifest/approved_packages.json`, outside the supplier's package directory. It records the name, version and SHA-256 digest of the approved wheel. The pipeline sets it read-only (chmod 444) before the build runs (Figure 11).

```
{
  "schema_version": "1.0",
  "packages": [
    {
```

```
      "name": "html_note_formatter",
      "version": "0.1.0",
      "algorithm": "sha256",
      "digest": "c78738054c6302873d424ea2c14ecd05a214140e9dfd3ba12e417be734ab0f55"
    }
  ]
}
```

```
(venv) (base) utkubalikci@MacBook-Pro-tilhrende-utku Software_sec_assigment1 %  ls -la manifest/
total 8
drwxr-xr-x   3 utkubalikci  staff   96 Oct 18 20:07 .
drwxr-xr-x@ 20 utkubalikci  staff  640 Oct 18 20:13 ..
-r--r--r--   1 utkubalikci  staff  463 Oct 18 20:07 approved_packages.json
```

Figure 11. *Manifest permissions (-r--r--r--) set before dependency verification: ls -la manifest/.*

7.3 Verifier and Gate

`scripts/verify_integrity.py` hashes the built wheel in 64 KB chunks and compares the digest with the manifest. A match exits with code 0 and allows installation; a mismatch exits with code 1 and stops the pipeline.

```
def compute_digest(algorithm: str, file_path: Path) -> str:
    h = hashlib.new(algorithm)
    with open(file_path, 'rb') as f:
        for chunk in iter(lambda: f.read(65536), b''):
            h.update(chunk)
    return h.hexdigest()
```

```
  Manifest : /Users/utkubalikci/Desktop/Software_sec_assigment1/manifest/approved_packages.json

  Package : html_note_formatter v0.1.0
  Artifact: /Users/utkubalikci/Desktop/Software_sec_assigment1/packages/html_note_formatter/dist/html_note_formatter-0.1.0-py3-none-any.wh
1
  Expected: c78738054c6302873d424ea2c14ecd05a214140e9dfd3ba12e417be734ab0f55
  Actual  : c78738054c6302873d424ea2c14ecd05a214140e9dfd3ba12e417be734ab0f55
  [PASS] Digest matches — artifact is authentic.

Integrity gate PASSED. Proceeding with installation.
```

Figure 12. *Integrity gate on the authentic artifact: digest matches the manifest and installation proceeds.*

7.4 Tamper Demonstration

`scripts/demo_tamper.py` runs three phases with the same filename and version: the authentic wheel, a tampered wheel, and the restored wheel. The tampered phase must fail with exit code 1.

Phase	SHA-256 (prefix)	Decision
1. Authentic	c7873805…ab0f55	PASS: installation proceeds
2. Tampered	053ec061…83ba56	BLOCKED: digest mismatch, exit code 1
3. Restored	c7873805…ab0f55	PASS: build proceeds

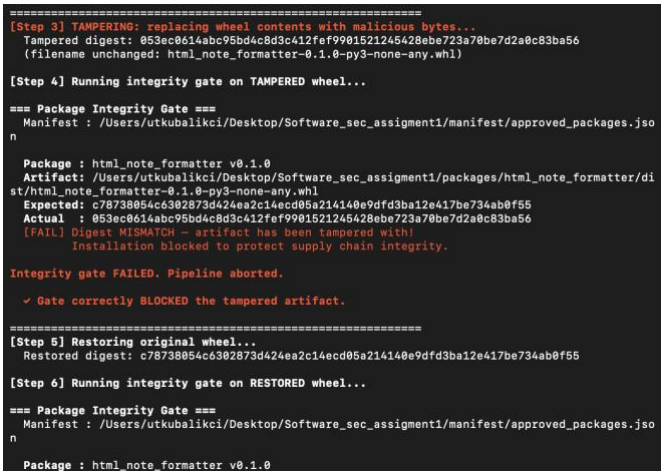


Figure 13: Terminal output from demo_tamper.py demonstrating the SHA-256 integrity gate (Experiment A) blocking a tampered wheel package and passing the restored artifact.

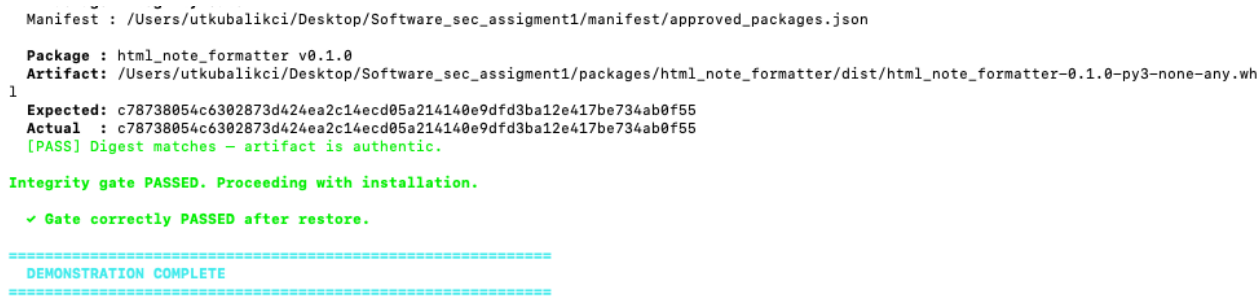


Figure 14. *Restored artifact: the gate passes again and the demonstration completes.*

7.5 Scope

Experiment A checks identity at install time. It does not limit what an approved package does once it runs.

8. What Would Change for Production

Moving to production would replace the in-memory store with a managed PostgreSQL database that encrypts data at rest and replicates it. Plain HTTP between services would become mutual TLS, with secrets held in a vault. The synchronous export would move to an asynchronous queue with sandboxed workers, using seccomp or AppArmor to contain the formatter. Rate limits, payload quotas and structured audit logs shipped to a SIEM would complete the picture.

9. Known Limitations

Fallback secret. The secure branch reads `JWT_SECRET_KEY` with a fixed default, so an unset variable silently reintroduces CWE-798. The fix is to refuse to start when the variable is missing.

Token scope. Tokens carry no audience or per-action scope. A valid token can reach any endpoint. Scoped claims would restrict each token to the services it needs.

Transport. Services talk plain HTTP on localhost, so an adjacent process could read the identity headers.

Denial of service. The export is synchronous and has no payload limit or rate limit.

Logging. Events go only to standard output, so repudiation is not mitigated.

Bandit gate. The static analysis stage cannot fail the build while `|| true` remains.

Runtime containment. Experiment B was not performed. Experiment A proves that a tampered package is refused at install time, not that an approved package is confined when it runs.

10. Conclusion

The system shows that microservices need layered, independent controls. Each service must check authority for the data it owns rather than trusting its neighbours. The planted vulnerabilities make the point concrete: the gateway alone does not protect the Processing Service, and a correct service can still leak data when composed with another. Experiment A shows that the build pipeline needs its own cryptographic check, because a version number can be reused for different bytes.

11. AI-Use Declaration

In line with Aarhus University's policy on generative AI (https://studerende.au.dk/en/gai), in this report Anthropic Claude and Google Antigravity was used during design, implementation and documentation. These tools drafted boilerplate route handlers, test scaffolding and parts of the STRIDE taxonomy, and helped format tables. I designed the architecture and the planted vulnerabilities, wrote the pipeline stages and test assertions, ran all experiments, and verified every result reported here.

References

1. Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. Proceedings of the IEEE, 63(9), 1278–1308.

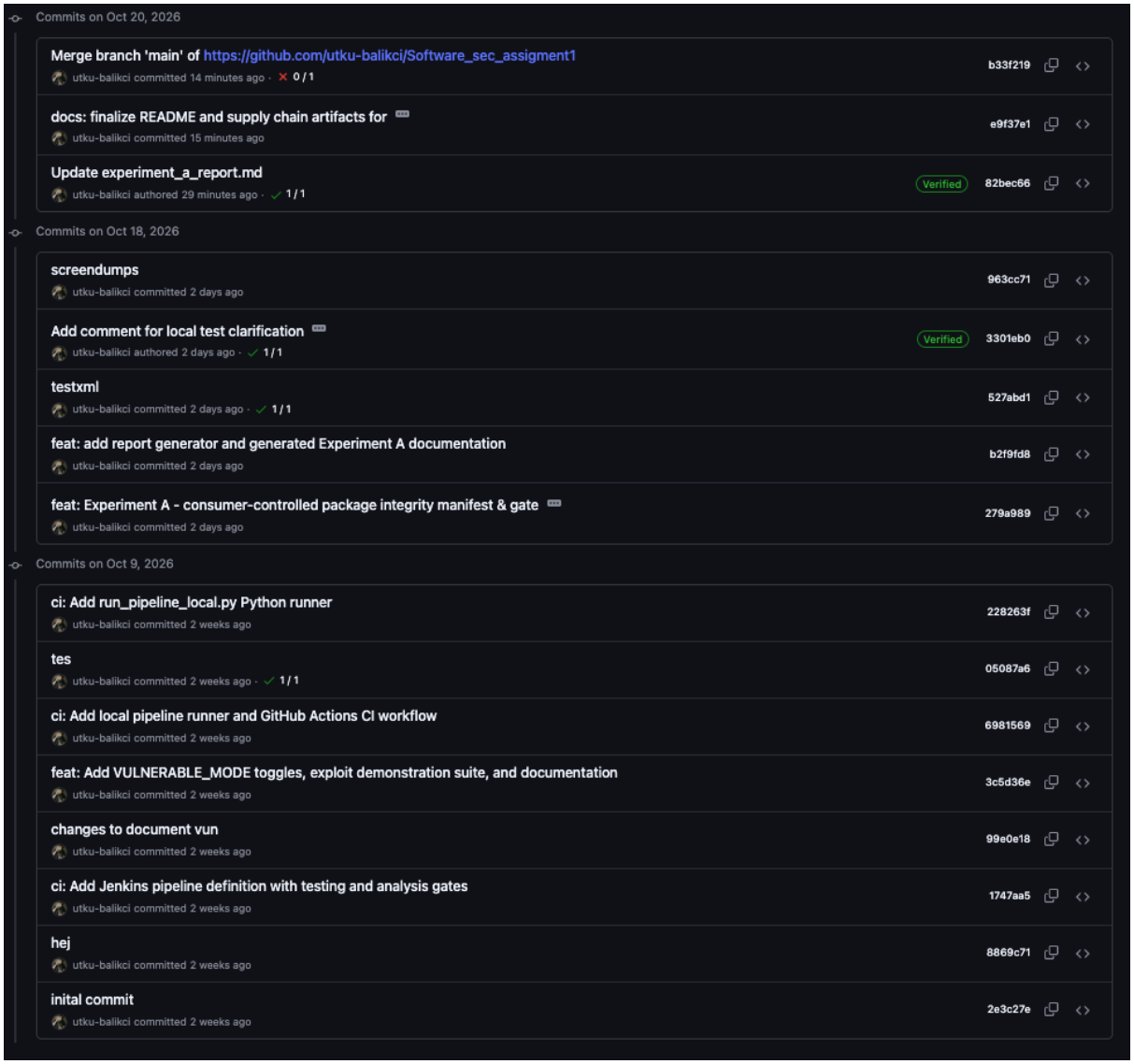
2. OWASP Foundation (2021). OWASP Top 10: Web Application Security Risks.

3. MITRE. Common Weakness Enumeration: CWE-798, CWE-862, CWE-209.

4. Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519.

5. NIST (2022). Secure Software Development Framework (SSDF) Version 1.1. SP 800-218.

Appendix A: Git History



Appendix B: Test Suite

```
tests/test_flow.py
  test_login_flow()
  test_functionality_1_create_note()
  test_functionality_2_view_note_and_authorization()
  test_functionality_3_export_note()
  test_admin_and_delete_permissions()

tests/test_vulnerabilities.py
  test_demonstrate_cwe_798_hardcoded_credentials()
  test_demonstrate_cwe_862_idor()
  test_demonstrate_cwe_209_sensitive_info_exposure()
  test_demonstrate_compositional_vulnerability()
```

Appendix C: Jenkinsfile

```
pipeline {
    agent any
    environment { PYTHONUNBUFFERED = '1' }
    stages {
```

```
        stage('Checkout & Setup') {
            steps { sh '''python3 -m venv venv && . venv/bin/activate && pip install fastapi uvicorn
requests pyjwt pytest build wheel''' }
        }
        stage('Package Integrity Gate') {
            steps {
                sh '''
                    chmod 444 manifest/approved_packages.json
                    . venv/bin/activate
                    cd packages/html_note_formatter && python -m build --wheel && cd ../..
                    python3 scripts/verify_integrity.py
                    pip install packages/html_note_formatter/dist/*.whl
                '''
            }
        }
        stage('Static Analysis & Security Gate') {
            steps { sh '. venv/bin/activate && bandit -r services/ -ll || true' }
        }
        stage('Launch Services & Integration Tests') {
            steps {
                sh '''
                    . venv/bin/activate
                    python run_services.py &
                    RUNNER_PID=$!
                    sleep 3
                    pytest -v tests/test_flow.py --junitxml=test-results.xml || TEST_FAILED=1
                    kill -TERM $RUNNER_PID || true
                    if [ "$TEST_FAILED" = "1" ]; then exit 1; fi
                '''
            }
        }
    }
    post {
        always {
            junit allowEmptyResults: true, testResults: 'test-results.xml'
            archiveArtifacts artifacts: 'test-results.xml, docs/*', fingerprint: true
        }
    }
}
```

Appendix D: Repository Layout

```
manifest/approved_packages.json         read-only, consumer-controlled
scripts/verify_integrity.py            integrity gate (exit 0 / 1)
scripts/demo_tamper.py                 tamper demonstration
scripts/generate_report.py             evidence generation
docs/experiment_a_report.md            generated Experiment A evidence
packages/html_note_formatter/dist/     supplier wheel
```

Appendix E: Reproduction

Run from the repository root with the virtual environment active:

- Secure mode: `python run_services.py`, then `pytest -v tests/test_flow.py`.
- Vulnerable mode: `python run_services.py --vulnerable`, then `pytest -s -v tests/test_vulnerabilities.py`.
- Static analysis: `bandit -r services/ -ll`.
- Integrity demonstration: `python3 scripts/demo_tamper.py`.
- Full local pipeline: `./run_pipeline_local.sh`.